

Linguagem D++ - Etapa 5

Tabela de Símbolos e Análise Semântica

Disciplina Compiladores - 2026.1 - Grupo 25

1. Visão geral

Esta etapa implementa a análise semântica da linguagem D++ sobre a árvore sintática abstrata já produzida pelo analisador SableCC construído nas Etapas 3 e 4 (`src/dplusplus.sable`). A análise foi implementada com o padrão de projeto **Visitor**, através de uma subclasse de `DepthFirstAdapter` (classe gerada automaticamente pelo SableCC), que percorre a árvore em profundidade e, a cada nó, aplica as verificações exigidas pela especificação da linguagem (Seção 4 do documento `D_v5.pdf`).

Todo o código novo desta etapa está no pacote `dplusplus.semantico` (`src/dplusplus/semantico/`), e um driver de linha de comando foi adicionado em `src/dplusplus/Main.java` para encadear léxico → sintático → semântico e imprimir o relatório de erros.

1.1 Os dois passos da análise

A análise semântica foi dividida em dois passos, executados em sequência sobre a mesma árvore:

Passo	Classe	Responsabilidade
1. Coleta de classes	<code>ColetorDeClasses</code>	Percorre o programa e monta a <code>TabelaClasses</code> completa: registra cada <code>family</code> , resolve a genealogia (herança), detecta ciclos/erros de herança e "achata" os atributos e métodos herdados em cada classe — tudo isso <i>antes</i> de qualquer corpo de método ser analisado.
2. Análise semântica	<code>AnalizadorSemantico</code>	O visitor propriamente dito (extends <code>DepthFirstAdapter</code>). Usa a <code>TabelaClasses</code> já pronta do passo 1 para checar escopo, inicialização e tipos em cada declaração, comando e expressão do corpo dos métodos.

O motivo de existirem dois passos é a resolução de referências "para frente": em D++ uma classe pode ter um atributo cujo tipo é outra classe declarada mais adiante no arquivo, e a herança precisa ser resolvida em ordem topológica (classe mãe antes da classe filha) para que os membros herdados fiquem visíveis corretamente, uma ordem que não corresponde necessariamente à ordem de visita em profundidade da árvore. Por isso o passo 1 é uma travessia direta do objeto (usando os getters gerados pelo SableCC), e não um visitor; o padrão Visitor/DepthFirstAdapter é usado no passo 2, que já encontra toda a informação de classes pronta.

2. Tabela de símbolos: hash table de listas encadeadas

A classe `TabelaSimbolos` (`src/dplusplus/semantico/TabelaSimbolos.java`) implementa uma hash table cujo tratamento de colisão é feito por encadeamento: cada posição do vetor é a cabeça de uma lista ligada de símbolos (classe interna `No`). Essa é a estrutura de dados pedida no enunciado, e **cada escopo da linguagem é uma instância diferente dessa classe**: cada classe, cada método (parâmetros) e cada bloco `start...finish` tem a sua própria `TabelaSimbolos`.

A função de hash é a tradução direta, para Java, do código em C fornecido nos slides:

```
#define SIZE 211      // tamanho padrão (mesmo valor clássico usado no livro do
Louden)
#define SHIFT 4

int hash(char* key)
{
    int temp = 0;
    int i = 0;
    while (key[i] != '\0')
    {
        temp = ((temp << SHIFT) + key[i]) % SIZE;
        ++i;
    }
    return temp;
}

private int hash(String chave) {
    int temp = 0;
    for (int i = 0; i < chave.length(); i++) {
        temp = ((temp << SHIFT) + chave.charAt(i)) % tamanho;
    }
    return temp;
}
```

2.1 PilhaEscopos: encadeamento entre escopos

A `TabelaSimbolos` resolve nomes *dentro* de um único escopo. Quem implementa a regra de visibilidade "se não está aqui, procure no escopo mais externo" é a classe `PilhaEscopos`: uma pilha de `TabelaSimbolos`. Entrar em uma classe, em um método ou em um bloco `start...finish` empilha uma nova tabela (`abrirEscopo()`); sair desses elementos desempilha (`fecharEscopo()`). A busca de um identificador (`buscar()`) percorre a pilha do topo (bloco mais interno) até a base, retornando o primeiro símbolo encontrado, implementando declaração/uso "no bloco onde a variável foi declarada", conforme pede a especificação da linguagem.

No fundo da pilha, quando se está dentro dos métodos de uma classe, fica empilhada a *própria* tabela de membros da classe (atributos + métodos, já com a herança "achatada", ver Seção 3). Isso implementa naturalmente a regra "posso chamar diretamente um atributo, método ou função que pertença ao meu objeto ou às minhas classes ancestrais": a busca cai, no pior caso, na tabela de membros da classe atual.

3. Classes, herança e polimorfismo

3.1 TabelaClasses e InfoClasse

`TabelaClasses` é o registro global de classes do programa (ela mesma também implementada sobre uma `TabelaSimbolos`, satisfazendo a mesma estrutura de dados para o escopo global de nomes de classe). Ela já nasce com as duas classes previstas na especificação: `Root` (topo da hierarquia; não pode ser instanciada; sem atributos/métodos próprios) e `Peripherals` (com `show[]` e `capture[]` pré-cadastrados).

Cada classe é representada por um `InfoClasse`, contendo o nome, a referência para a classe mãe e uma `TabelaSimbolos` de membros. Essa tabela de membros é construída "**achatada**": ao processar uma classe filha, primeiro copiam-se (clonam-se) todos os símbolos da tabela de membros da classe mãe (já resolvida, por construção topológica) e, em seguida, inserem-se os membros declarados na própria classe, um método com o mesmo nome de um método herdado *sobrescreve* a versão herdada na tabela, desde que a assinatura seja compatível (mesmos tipos de parâmetro e retorno igual ou covariante — ver 3.2). O resultado é que cada classe enxerga, em uma única tabela, tudo o que ela própria declarou mais tudo o que herdou, sem precisar percorrer a cadeia de ancestrais a cada consulta.

Classes que não aparecem como filhas em nenhuma relação de genealogia recebem `Root` como mãe implícita. A resolução da herança detecta e reporta: classe mãe não declarada, mais de uma mãe para a mesma classe (herança simples é obrigatória), classe derivando de si mesma e ciclos de herança entre duas ou mais classes (nesses casos de erro, a classe problemática recebe `Root` como mãe de segurança, para que o restante da análise continue sem lançar exceções).

3.2 Polimorfismo e sobrescrita

A compatibilidade de tipos para classes é feita percorrendo a cadeia de mães: um valor do tipo `Filha` é aceito onde se espera o tipo `Mae` sempre que `Filha` for, direta ou indiretamente, descendente de `Mae` (`Tipo.atribuivel / TabelaClasses.ehSubtipoOuIgual`). Isso vale para: inicialização de atributos/variáveis do tipo objeto, atribuição (`id << exp`), passagem de argumentos e retorno de função — cobrindo o requisito de suporte a polimorfismo nas checagens de tipo.

Como D++ não permite sobrecarga, uma classe não pode ter dois membros com o mesmo nome. Ao herdar um método e redeclará-lo na classe filha (sobrescrita), a nova assinatura precisa ter exatamente os mesmos tipos de parâmetro; o tipo de retorno pode ser o mesmo tipo ou um subtipo dele (retorno covariante), o que também é uma forma de polimorfismo. Qualquer outra diferença é reportada como erro (arquivo de teste `erro_10_sobrescrita_incompativel.dpp`).

3.3 Métodos abstratos

Um método sem corpo (`procedure_header dot / function_header dot`) é marcado `abstrato` no símbolo correspondente. Uma classe é considerada abstrata se, depois de achatada a herança, qualquer entrada de método na sua tabela de membros ainda estiver marcada como abstrata, ou seja, uma classe filha só é concreta se sobrescreveu (com corpo) todos os métodos abstratos herdados, exatamente como pede o enunciado. Classes abstratas (e `Root`) não podem ser instanciadas.

4. Resumo das checagens implementadas

Requisito do enunciado	Onde/como foi implementado
Entidades inseridas na tabela de símbolos com os atributos necessários ao serem declaradas	<code>ColetorDeClasses</code> insere classes, atributos e métodos (com tipo, parâmetros, retorno e flag de abstrato); <code>AnalizadorSemantico</code> insere objetos/variáveis/constantes/parâmetros locais em <code>outAObjetoAtributosAlt</code> , <code>outAVariavelAtributosAlt</code> , <code>outAConstanteAtributosAlt</code> e <code>outAParametroParametro</code> .
Entidade declarada e em escopo válido no momento do uso	<code>PilhaEscopos.buscar</code> , usado em toda referência a identificador (<code>outAAtributoExp</code> , <code>outAChamadaChamada</code> via <code>resolverAcesso</code> , e no alvo de <code>outAAtribuicaoComando</code>); reporta "não foi declarado" quando a busca falha em toda a pilha de escopos.
Entidades declaradas/definidas (inicializadas) quando necessário	Campo <code>inicializado</code> em <code>Simbolo</code> , checado em todo uso e atualizado em toda atribuição.
Compatibilidade de tipos em declarações, atribuições, chamadas e expressões (com polimorfismo e herança)	Classe <code>Tipo</code> (<code>atribuivel/unificar</code>), usada nas declarações (<code>outAVariavelAtributosAlt/outAConstanteAtributosAlt</code>), nas atribuições (<code>outAAtribuicaoComando</code>), nos argumentos de chamada (<code>outAChamadaChamada</code>), no retorno de função (<code>outAFuncaoMetodosAlt</code>) e em cada operador de expressão (<code>outAPlusExp</code> , <code>outAEqExp</code> , <code>outATernarioExp</code> , etc.).
Ponto de entrada único (>>)	Contado durante o passo 1; erro se for zero ou mais de um

Requisito do enunciado	Onde/como foi implementado
	(<code>erro_5_sem_ponto_de_entrada.dpp</code> , <code>erro_6_multiplos_pontos_de_entrada.dpp</code>). Também é erro se a classe que contém o ponto de entrada for abstrata (não poderia ser instanciada automaticamente pela "execução fictícia" descrita no enunciado).
Herança simples, classe Root implícita, métodos abstratos/classes concretas	Seção 3 deste relatório.
Sem sobrecarga de operadores/funções; sem variáveis compostas	Sem sobrecarga: um nome só pode ter uma entrada por classe (chechado em <code>ColetorDeClasses.inserirAtributo/inserirOuSobrescreverMetodo</code>). Sem variáveis compostas: a gramática da Etapa 4 já não oferece nenhuma construção de tipo composto, então não há o que checar aqui além dos dois tipos primitivos e dos tipos de classe.
Prioridade de operadores igual à de C	Já garantida pela gramática livre de contexto da Etapa 3/4 (precedência codificada na própria hierarquia de produções <code>exp_or/exp_and/exp_rel/exp_add/exp_mul/exp_unario</code>).
Parâmetros passados por cópia	Cada parâmetro é um <code>Simbolo</code> independente no escopo do método (<code>Categoria.PARAMETRO</code>); reatribuí-lo dentro do método não afeta nada fora dele, pois não existe qualquer mecanismo de referência/alias para parâmetros na análise.
Classe <code>Peripherals</code> com <code>show[]/capture[]</code>	Pré-cadastrada em <code>TabelaClasses</code> , com um tipo especial <code>Tipo.PRIMITIVO_COMPATIVEL</code> que "casa" tanto com <code>number</code> quanto com <code>answer</code> , conforme pedido no enunciado ("assumam que os dois métodos são compatíveis com os tipos primitivos").

5. Mensagens de erro

Em vez de abortar no primeiro problema, `GerenciadorErros` acumula todas as mensagens encontradas (cada uma com linha/coluna, quando aplicável) e as imprime ordenadas ao final da análise.

6. Como executar

```
javac -d bin $(find src/dplusplus -name "*.java")
java -cp bin dplusplus.Main "caminho/para/arquivo.dpp"
```

```
# a flag --tabela também imprime, ao final, um resumo da tabela de classes montada:  
java -cp bin dplusplus.Main "caminho/para/arquivo.dpp" --tabela
```